

CMPSC 448 Midterm Project: Who Wrote It? Identifying LLMs from Their Responses

Team leader: Aryan Desai (individual project)

GitHub: <https://github.com/Desai0Aryan/who-wrote-it>

1. Introduction

Large Language Models (LLMs) such as ChatGPT, Claude, and Gemini are being used to create more and more text. This made me interested in whether we can tell which LLM wrote a response just by looking at the response.

For this project, I created a dataset of responses from three different LLMs: ChatGPT, Claude, and Gemini. I then trained machine learning models to predict which LLM created each response.

The two main research questions were:

- RQ1: Can we identify which LLM generated a response?
- RQ2: Does the user's prompt help identify the LLM?

I also looked at two extra questions:

- RQ3: Can the models identify LLMs when they are tested on a type of task they did not see during training?
- RQ4: What features of the responses make the LLMs easier to tell apart?

2. Dataset

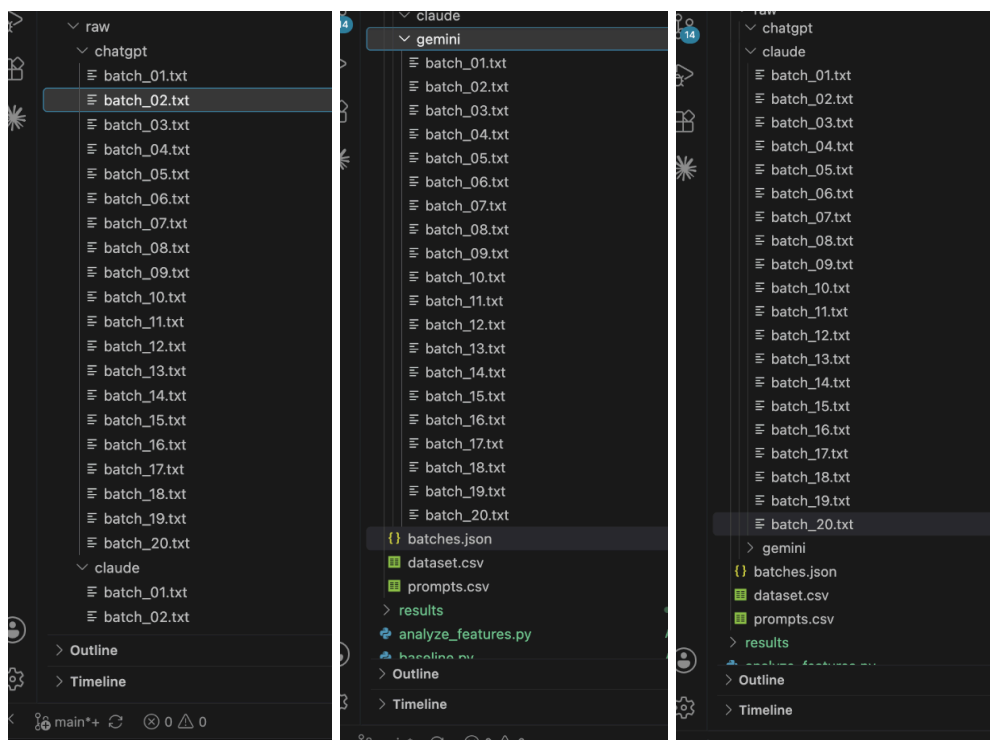
I created 120 prompts for the project. The prompts were split into four categories:

- 30 coding questions
- 30 math questions
- 30 factual questions
- 30 writing questions

I gave all 120 prompts to each of the three LLMs.

This gave me: $120 \text{ prompts} \times 3 \text{ LLMs} = 360 \text{ total responses}$.

This also made the dataset balanced because each LLM had the same number of responses. The data included the original prompt, the response, the task category, and the name of the LLM that created the response. I collected the responses from the free versions of ChatGPT, Claude, and Gemini and saved them into CSV files for the experiments. I also made sure that responses from the same prompt stayed together when splitting the data. This was important because otherwise the model could see the same question during training and testing and get an unfairly high score.



The main limitation of the dataset is that it is not very large. I only had 360 responses, and I only tested three LLMs. A larger dataset with more models would make the results stronger.

3. Models and Training

I used the two models required by the assignment:

- Convolutional Neural Network (CNN)
- Recurrent Neural Network using a BiLSTM

I also created a TF-IDF + Logistic Regression model as a baseline.

CNN: The CNN takes the text and turns it into word/token embeddings. It then uses convolution layers to find patterns in the text. The outputs are combined and passed through a final layer that predicts one of the three LLMs.

BiLSTM: The BiLSTM reads the text in both directions. This lets it use information from earlier and later parts of the response when making its prediction. Both neural networks used the same dataset splits so that the results could be compared fairly. I trained the models using the Adam optimizer and cross-entropy loss. I also used dropout and early stopping to help prevent overfitting. The baseline model used TF-IDF features with Logistic Regression. I included this because I wanted to see if a simpler machine learning model could perform as well as the neural networks.

```
rq1 full mode=output lstm seed=0 fold=0 acc=0.536 (4s)
saved results/preds_rq1_nn_full_quick.csv
(.venv) aryandesai@Aryans-MacBook-Pro who_wrote_it % python train_nn.py --exp rq1
device=cpu exp=rq1 variant=full models=['cnn', 'lstm'] modes=['output']
rq1 full mode=output cnn seed=0 fold=0 acc=0.764 (33s)
rq1 full mode=output cnn seed=0 fold=1 acc=0.736 (58s)
rq1 full mode=output cnn seed=0 fold=2 acc=0.792 (38s)
rq1 full mode=output cnn seed=0 fold=3 acc=0.792 (29s)
rq1 full mode=output cnn seed=0 fold=4 acc=0.833 (23s)
rq1 full mode=output cnn seed=1 fold=0 acc=0.806 (27s)
rq1 full mode=output cnn seed=1 fold=1 acc=0.667 (19s)
rq1 full mode=output cnn seed=1 fold=2 acc=0.764 (22s)
rq1 full mode=output cnn seed=1 fold=3 acc=0.833 (36s)
rq1 full mode=output cnn seed=1 fold=4 acc=0.806 (37s)
rq1 full mode=output cnn seed=2 fold=0 acc=0.819 (26s)
rq1 full mode=output cnn seed=2 fold=1 acc=0.778 (44s)
rq1 full mode=output cnn seed=2 fold=2 acc=0.806 (53s)
rq1 full mode=output cnn seed=2 fold=3 acc=0.667 (25s)
rq1 full mode=output cnn seed=2 fold=4 acc=0.819 (26s)
rq1 full mode=output lstm seed=0 fold=0 acc=0.639 (28s)
rq1 full mode=output lstm seed=0 fold=1 acc=0.681 (42s)
rq1 full mode=output lstm seed=0 fold=2 acc=0.708 (33s)
rq1 full mode=output lstm seed=0 fold=3 acc=0.653 (33s)
rq1 full mode=output lstm seed=0 fold=4 acc=0.639 (44s)
rq1 full mode=output lstm seed=1 fold=0 acc=0.667 (24s)
rq1 full mode=output lstm seed=1 fold=1 acc=0.694 (26s)
rq1 full mode=output lstm seed=1 fold=2 acc=0.792 (43s)
rq1 full mode=output lstm seed=1 fold=3 acc=0.708 (26s)
rq1 full mode=output lstm seed=1 fold=4 acc=0.653 (38s)
rq1 full mode=output lstm seed=2 fold=0 acc=0.653 (22s)
rq1 full mode=output lstm seed=2 fold=1 acc=0.694 (26s)
rq1 full mode=output lstm seed=2 fold=2 acc=0.736 (34s)
rq1 full mode=output lstm seed=2 fold=3 acc=0.708 (21s)
rq1 full mode=output lstm seed=2 fold=4 acc=0.639 (21s)
saved results/preds_rq1_nn_full.csv
(.venv) aryandesai@Aryans-MacBook-Pro who_wrote_it %
```

4. RQ1: Can We Identify the LLM?

For RQ1, I only gave the models the LLM response and asked them to predict whether it came from ChatGPT, Claude, or Gemini. Since there are three possible answers, random guessing would give about 33.3% accuracy.

The results were:

Model	Accuracy
CNN	77.9%
BiLSTM	68.4%
TF-IDF + Logistic Regression	85.6%

All three models performed much better than random guessing. The best model was actually the simple TF-IDF + Logistic Regression model, with 85.6% accuracy. CNN was the best of the two required neural networks at 77.9%. This was interesting because I expected the neural networks to perform better than the simpler baseline. The results showed me that a more complicated model is not always better, especially when the dataset is relatively small.

```

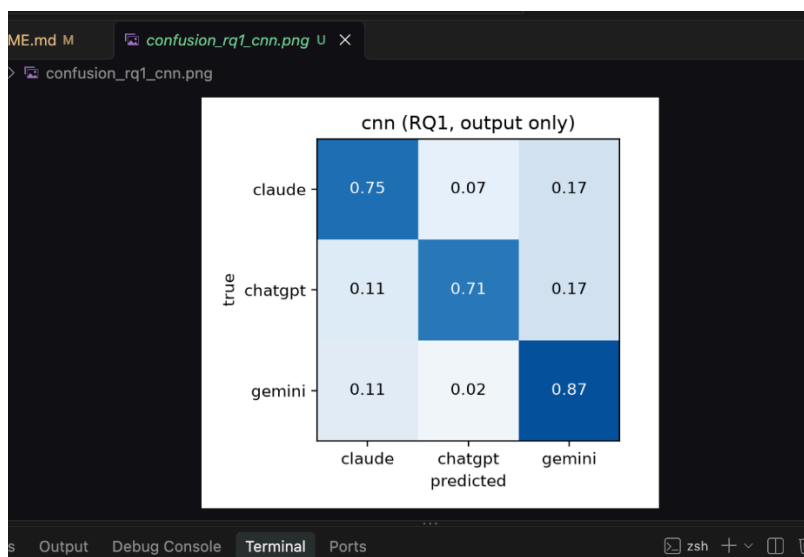
(.venv) aryandesai@Aryans-MacBook-Pro who_wrote_it % python summarize.py

=== RQ1 (mean/std over CV folds) ===
variant model mode acc_mean acc_std f1_mean f1_std n_folds
full cnn output 0.779 0.035 0.776 0.037 5.0
full lstm output 0.684 0.040 0.680 0.042 5.0
full tfidf_lr output 0.856 0.051 0.855 0.050 5.0
nofmt cnn output 0.705 0.036 0.698 0.038 5.0
nofmt lstm output 0.566 0.066 0.555 0.068 5.0

=== RQ2 (mean/std over CV folds) ===

```

I also created confusion matrices to see which LLMs the models confused with each other.



5. RQ2: Does the Prompt Help?

For RQ2, I tested three different types of input:

1. Prompt only
2. Response only
3. Prompt + response

The results were:

Model	Prompt Only	Response Only	Prompt + Response
CNN	33.3%	77.9%	79.4%
BiLSTM	33.3%	68.4%	52.5%

TF-IDF	33.3%	85.6%	84.2%
--------	-------	-------	-------

The prompt-only results were exactly 33.3%, which is the same as random guessing. This makes sense because I gave the same prompts to all three LLMs. The prompt itself does not tell us which model answered it. The response was much more useful for identifying the LLM. Adding the prompt slightly helped the CNN, but it actually hurt the BiLSTM and TF-IDF models. Overall, the response was more important than the prompt.

```

=== RQ2 (mean/std over CV folds) ===
variant model mode acc_mean acc_std f1_mean f1_std n_folds
full cnn both 0.794 0.037 0.793 0.040 5.0
      cnn input 0.333 0.000 0.296 0.017 5.0
      cnn output 0.779 0.035 0.776 0.037 5.0
      lstm both 0.525 0.044 0.519 0.049 5.0
      lstm input 0.333 0.000 0.297 0.015 5.0
      lstm output 0.684 0.040 0.680 0.042 5.0
      tfidf_lr both 0.842 0.040 0.842 0.040 5.0
      tfidf_lr input 0.333 0.000 0.167 0.000 5.0
      tfidf_lr output 0.856 0.051 0.855 0.050 5.0

```

6. RQ3: Does It Work on Different Tasks?

For the extra-credit RQ3 experiment, I tested whether a model could identify an LLM on a type of task that it had not seen during training. For example, the model could be trained on coding, math, and factual questions and then tested on writing questions.

The results were lower than the normal RQ1 results.

Model	Average Accuracy
CNN	49.5%
BiLSTM	42.6%
TF-IDF	53.9%

This shows that identifying an LLM becomes harder when the model is tested on a new type of task. The results suggest that the classifier is learning some general differences between the LLMs, but it is also learning patterns related to the type of task.

```

=== RQ3 accuracy when the listed task is held out of training ===
fold      coding factual  math  writing  mean
variant model
full      cnn      0.463   0.522  0.552   0.441  0.495
          lstm     0.415   0.381  0.467   0.441  0.426
          tfidf_lr  0.500   0.600  0.522   0.533  0.539

```

7. RQ4: What Makes the LLMs Different?

For RQ4, I looked at different features of the responses to see what could help identify the LLM. Some of the features I looked at were:

- Response length
- Number of lines
- Formatting
- Bullet points
- Headers
- Code blocks
- LaTeX
- Vocabulary
- Punctuation

One clear difference was response length. Claude responses were longer on average than ChatGPT and Gemini responses. I also tested what happened when I removed formatting from the responses. For the CNN, accuracy dropped from 77.9% to 70.5%. For the BiLSTM, accuracy dropped from 68.4% to 56.6%. This suggests that formatting is an important clue that the models use. I also tested different groups of features. Length and formatting were more useful than some of the other features. This means that the models may not only be learning differences in word choice. They may also be learning things like how long an answer is and how it is formatted.

```
• (.venv) aryandesai@Aryans-MacBook-Pro who_wrote_it % python analyze_features.py
```

```
=== A. Mean stylometric features per LLM ===
```

llm	claude	chatgpt	gemini
n_chars	907.633	603.525	699.483
n_words	138.908	86.567	101.417
n_lines	11.467	12.350	8.867
blank_lines	4.383	5.175	3.992
bold_count	2.583	2.275	2.433
header_count	0.042	0.142	0.292
bullet_lines	1.417	1.217	1.983
code_blocks	0.400	0.408	0.258
latex	0.117	4.692	5.683
avg_word_len	4.446	4.687	4.807
ttr	0.652	0.649	0.646
avg_sent_len	22.307	24.170	28.637
em_dash	0.008	0.300	0.217
exclaim	0.242	0.183	0.242
question	0.108	0.117	0.083
commas_per_word	0.086	0.073	0.063
colon_count	4.000	2.950	4.083
digit_frac	0.037	0.027	0.020
emoji	0.075	0.017	0.025

```
Median response length (words) by LLM x task:
```

category	coding	factual	math	writing
llm_name				
chatgpt	66.0	129.0	27.0	104.0
claude	134.0	216.0	20.0	156.0
gemini	99.0	138.0	50.0	107.0

```
=== B. Random forest on handcrafted features (5-fold CV, chance = 0.333) ===
```

features	acc_mean	acc_std
all features	0.744	0.044
drop length	0.669	0.055
drop formatting	0.672	0.041
drop lexical	0.742	0.035
drop punctuation	0.719	0.057
only length	0.611	0.042
only formatting	0.525	0.098
only lexical	0.422	0.074
only punctuation	0.439	0.054

```
=== C. TF-IDF + LR on ablated text (5-fold CV) ===
```

text variant	acc_mean	acc_std
full text	0.856	0.045
no markdown/newlines	0.767	0.039
no punctuation, letters+digits only	0.731	0.040
first 30 tokens only	0.714	0.034
last 30 tokens only	0.694	0.056

```
=== D. Most indicative tokens per LLM (largest LR weights) ===
```

```
claude : '* \n', '!=', 'x', ' and', '" t"', '"', '= *', ' _', '" s"', '"', ' so', 'one', '/', '3', '\n -'  
chatgpt : ' \n', ' ', '\\', '\n $', '$ \n', '$ $', '- *', '\\ \n', '\n \\'', '\n -', 'boxed', 'boxed {', '\\ boxed',  
' } \n', '*', '  
gemini : '\n *', '$', 'here is', 'here', ': *', ' ) $', '\\ times', 'using', 'python', 'to', '$ (', '\n #', 'is a',  
'# #', '$ \\'
```

```
• (.venv) aryandesai@Aryans-MacBook-Pro who_wrote_it % python summarize.py
```

```

=== RQ1 accuracy by task type ===
category      coding factual  math  writing
variant model
full   cnn      0.878   0.633  0.959   0.644
      lstm      0.785   0.493  0.778   0.681
      tfidf_lr   0.889   0.811  0.978   0.744
nofmt  cnn      0.693   0.548  0.952   0.626
      lstm      0.715   0.389  0.722   0.437

=== RQ1 recall by LLM ===
true_llm      chatgpt  claude  gemini
variant model
full   cnn      0.714   0.753   0.869
      lstm      0.661   0.544   0.847
      tfidf_lr   0.742   0.950   0.875
nofmt  cnn      0.506   0.753   0.856
      lstm      0.542   0.439   0.717
saved results/confusion_rq1_cnn.png
saved results/confusion_rq1_lstm.png
saved results/confusion_rq1_tfidf_lr.png
❖ (.venv) aryandesai@Aryans-MacBook-Pro who_wrote_it % 

```

8. Lessons Learned

This project taught me that the way a dataset is created is very important.

One of the biggest things I learned was that a simple model can sometimes beat a more complicated model. The TF-IDF model had the highest accuracy even though the CNN and BiLSTM were more complicated.

I also learned why it is important to split the data correctly. Keeping responses from the same prompt together helped prevent the model from getting an unfair advantage.

Another thing I learned was that machine learning models can use simple patterns that I did not expect. In this project, things like response length and formatting were useful for identifying the LLM.

The RQ3 experiment also showed me that a model can perform much worse when it sees a type of task that it did not see during training.

If I continued this project, I would collect more responses, add more LLMs, and use more types of questions. This would make the results more reliable.

9. Conclusion

In this project, I tested whether machine learning could identify which LLM created a response. The results show that it is possible to identify the LLM better than random guessing. The best overall model was TF-IDF + Logistic Regression with 85.6% accuracy, while the CNN reached 77.9% and the BiLSTM reached 68.4%.

The prompt by itself was not useful because the same prompts were given to every LLM. The response contained much more information for identifying the model.

The extra experiments also showed that task type, response length, and formatting can affect how easy an LLM is to identify.

Overall, I learned that LLMs can have patterns in the way they write, but identifying these patterns is not always simple. A larger and more diverse dataset would be needed to build a more reliable LLM detector.